

TROUBLESHOOTING

ANDREI ARNALDO

BSIS 2

SCENARIO 1

FIXED CODE:

```
$conn = mysqli_connect("localhost", "root", "", "class_db");
```

```
$id = $_GET['id'];
```

```
$sql = "SELECT * FROM students WHERE student_id = $id";
```

```
$res = mysqli_query($conn, $sql);
```

```
$r = mysqli_fetch_assoc($res);
```

```
echo $r['first_name'];
```

EXPLANATION:

Gumagamit yung code ng **POST**, pero yung value galing sa **URL**.

Kapag nasa URL yung data (example: page.php?id=5), **GET** talaga ang gamit, hindi POST.

Kaya nagkakaroon ng **undefined index**, kasi **walang laman yung POST**, so hindi niya mahanap yung value.

SCENARIO 2

FIXED CODE:

```
$conn = mysqli_connect("localhost", "root", "", "class_db");
```

```
$fname = $_POST['fname'];
```

```
$sql = "SELECT * FROM students WHERE first_name = '$fname'";
```

```
$res = mysqli_query($conn, $sql);
```

EXPLANATION:

The first name is text, but walang quotes yung query. MySQL thinks the text is a column name, kaya error. Adding quotes makes MySQL treat it as a string.

SCENARIO 3

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");
```

```
$age = $_GET['age'];
```

```
$stmt = $conn->prepare("SELECT * FROM students WHERE age = ?");
```

```
$stmt->bind_param("i", $age);
```

```
$stmt->execute();
```

```
$res = $stmt->get_result();
```

EXPLANATION:

Yung code diretso nilalagay yung user input sa SQL, kaya unsafe siya. Puwede kasi may mag-type ng malicious commands. Ginagamit ang prepared statements para isecured yung query at pigilan yung SQL injection.

SCENARIO 4:

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");
```

```
$first = $_POST['fname'];
```

```
$last = $_POST['lname'];
```

```
if(empty($first) || empty($last)){
```

```
die("Fields cannot be empty");  
}
```

```
$sql = "INSERT INTO students (first_name,last_name) VALUES ('$first','$last')";  
mysqli_query($conn, $sql);
```

```
echo "Inserted!";
```

EXPLANATION:

The code inserts data kahit yung fields ay empty. This creates blank or invalid records. Checking the input first prevents errors.

SCENARIO 5:

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");
```

```
$email = $_POST['email'];
```

```
$sql = "SELECT * FROM students WHERE email='$email'";
```

```
$res = mysqli_query($conn, $sql);
```

EXPLANATION:

Yung variable name is misspelled (emial). PHP cannot find it and shows an undefined index.

SCENARIO 6:

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");
```

```
$id = intval($_GET['id']);
```

```
$sql = "DELETE FROM students WHERE student_id = $id";  
mysqli_query($conn, $sql);
```

EXPLANATION:

Direktang nilalagay yung GET input in SQL. A user can enter harmful commands. Using intval() or prepared statements makes it safe.

SCENARIO 7:

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");  
  
$id = $_POST['id'];  
$email = $_POST['email'];  
  
$sql = "UPDATE students SET email='$email' WHERE student_id=$id";  
$res = mysqli_query($conn, $sql);  
  
if(!$res){  
    die("SQL Error: " . mysqli_error($conn));  
}  
  
echo "Updated!";
```

EXPLANATION:

Yung SQL walang quotes around the email, kaya nag error. The code still prints “Updated!” because it does not check the result.

SCENARIO 8:

FIXED CODE:

```
$conn = mysqli_connect("localhost","root","","class_db");
```

```
$res = mysqli_query($conn,"SELECT * FROM students");
```

```
while($row = mysqli_fetch_assoc($res)){  
    echo $row['email'] . "<br>";  
}
```

EXPLANATION:

Yung code kumukuha lang ng isang row. Kaya isa lang ang email na lumalabas. Kapag gumamit ka ng loop, maiprint lahat ng records.

SCENARIO 9:

FIXED CODE:

```
$id = $_GET['id'];
```

EXPLANATION:

Links always send GET, not POST. Kaya yung POST nagiging empty and nagkakaroon ng undefined index. Using GET matches how the data is sent.

SCENARIO 10:

FIXED CODE:

```
$age = $_POST['age'];  
$sql = "SELECT * FROM students WHERE age = $age";
```

EXPLANATION:

Gumamit ng \$aeg instead na \$age. This causes an undefined variable error.

SCENARIO 11:

FIXED CODE:

PHP

```
$email = $_GET['email'];
```

HTML

```
<form method="POST" action="save.php">
```

EXPLANATION:

The form uses GET, but PHP expects POST. This causes an undefined index error. They must match.

SCENARIO 12:

FIXED CODE:

```
$id = intval($_GET['id']);  
  
$sql = "SELECT * FROM students WHERE student_id = $id";
```

EXPLANATION:

Yung numbers hindi dapat nasa labas ng quotes. Gumagana but not efficient and pwedeng magkaroon ng wrong indexing. Removing quotes is better.

SCENARIO 13:

FIXED CODE:

```
$newEmail = $_POST['email'];  
  
$id = $_POST['id'];  
  
$sql = "UPDATE students SET email='$newEmail' WHERE student_id=$id";  
  
mysqli_query($conn,$sql);
```

EXPLANATION:

Yung UPDATE naapektuhan lahat ng rows because there is no WHERE.

SCENARIO 14:

FIXED CODE:

```
$data = $_POST;
```

```
$sql = "INSERT INTO students (first_name, last_name, email)
VALUES ('{$data['first_name']}', '{$data['last_name']}', '{$data['email']}')";
```

EXPLANATION:

Array keys must have quotes, or PHP will show undefined index errors. Values also kaylangan ng quotes in SQL.

SCENARIO 15:

FIXED CODE:

```
$page = intval($_GET['page']);
```

```
if($page < 0) $page = 0;
```

```
$limit = 5;
```

```
$offset = $page * $limit;
```

```
$sql = "SELECT * FROM students LIMIT $offset, $limit";
```

EXPLANATION:

Users can enter a huge page number and cause errors or slowdowns. fix it by restricting the value. This keeps the system safe.